

UP LT GRADE • COMPUTER SCIENCE MAINS

Computer Graphics Complete Descriptive-Exam Notes

Simple, Clear, Exam-Ready — With Diagrams

Displays: Types of Computer Graphics • Random & Raster Scan Displays • Frame Buffer & Video Controller

Rasterization: Line Generating Algorithms • Circle Generating Algorithms

2D/3D Geometry: Transformations • Windowing & Clipping • Three-Dimensional Graphics

Rendering: Curves & Surfaces • Hidden Line & Surface Removal

Prepared for descriptive-exam revision • 45-day study plan

Table of Contents

1. Types of Computer Graphics	Page 3
2. Graphic Displays — Random Scan vs Raster Scan	Page 4
3. Frame Buffer & Video Controller	Page 6
4. Line Generating Algorithms (DDA & Bresenham's)	Page 7
5. Circle Generating Algorithm (Midpoint Circle)	Page 9
6. 2D Transformations	Page 10
7. Windowing and Clipping	Page 12
8. Three-Dimensional Graphics	Page 14
9. Curves and Surfaces	Page 16
10. Hidden Line and Surface Removal	Page 17

1. Types of Computer Graphics

In Simple Words

Computer Graphics is the field of creating, manipulating, and displaying images using a computer. It is broadly split into two categories based on how much the user can interact with the image.

Two Broad Categories

- **Interactive Computer Graphics:** The user can interact with and modify the image in real time (using a mouse, keyboard, or stylus) — e.g., CAD software, video games. Requires a two-way communication between user and computer.
- **Non-Interactive (Passive) Computer Graphics:** The image is generated once and the user has no control over it afterward — e.g., a computer-generated movie scene or a static chart. Only a one-way communication (computer to display).

Classification by Dimension

- **2D Graphics:** Images defined using only X and Y coordinates — e.g., logos, icons, 2D games.
- **3D Graphics:** Images defined using X, Y, and Z coordinates, allowing depth, rotation, and realistic rendering — e.g., animated films, CAD models.

Applications of Computer Graphics

- Computer-Aided Design (CAD) — engineering and architectural design
- Presentation Graphics — charts, graphs for reports
- Entertainment — movies, video games, animation
- Education and Training — simulators, visualizations
- Graphical User Interfaces (GUIs) — icons, windows, menus
- Image Processing — enhancing or analyzing existing images

Exam Tip

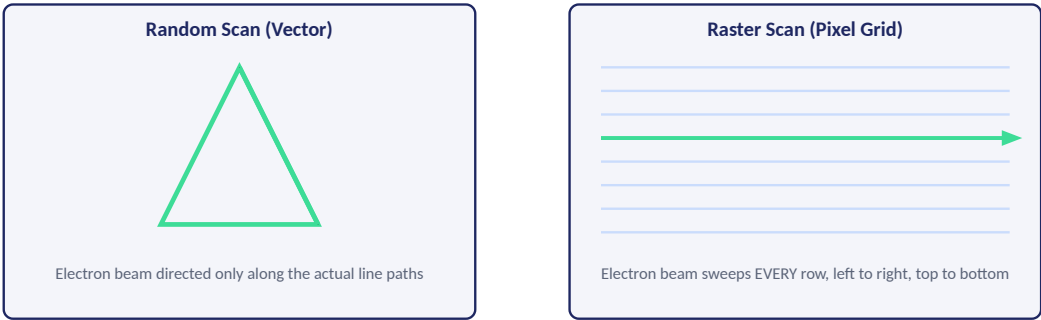
When asked to "classify computer graphics," always give BOTH classifications (Interactive vs Non-Interactive, AND 2D vs 3D) since questions often don't specify which axis they mean — covering both avoids losing marks to ambiguity.

2. Graphic Displays — Random Scan vs Raster Scan

In Simple Words

Display devices draw images in one of two fundamentally different ways: Random Scan draws only the actual lines/shapes needed (like tracing with a pen), while Raster Scan draws EVERY pixel on the screen in a fixed left-to-right, top-to-bottom pattern (like a typewriter).

Diagram: Random Scan vs Raster Scan



Comparison Table

Aspect	Random Scan	Raster Scan
Drawing Method	Beam directed only along the lines of the picture	Beam scans the ENTIRE screen row by row, regardless of picture
Resolution	Very high (limited only by beam precision)	Limited by number of pixels (fixed resolution)
Storage	Stores the picture as a set of line-drawing commands (Display File)	Stores the picture as pixel intensities in a Frame Buffer
Solid Areas / Shading	Difficult — mainly suited for line drawings (wireframes)	Easy — can fill regions with any color/shade
Cost	Expensive for complex/detailed images	Cost is independent of image complexity (fixed pixel grid)
Common Use	Older vector monitors, plotters	Modern TVs, monitors, phone screens

Exam Tip

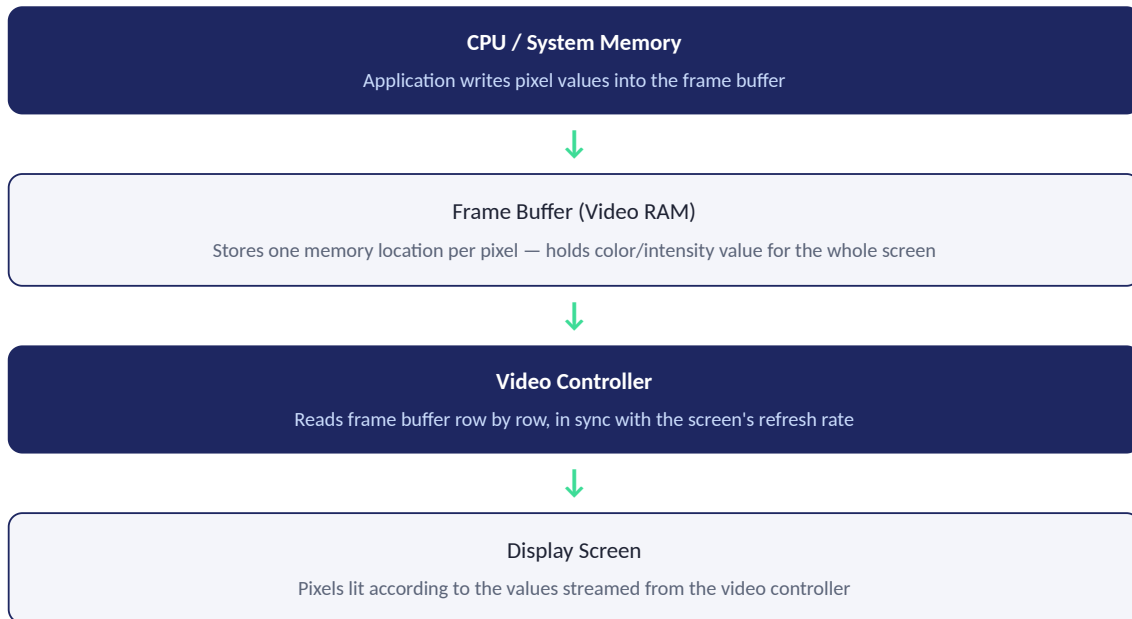
Always mention that modern displays are almost universally Raster Scan, and connect this topic directly to the NEXT topic (Frame Buffer) — raster scan systems require a frame buffer, while random scan systems use a display file instead.

3. Frame Buffer & Video Controller

In Simple Words

The Frame Buffer is a block of memory that stores the color/intensity value for EVERY pixel on the screen. The Video Controller is the hardware that continuously reads this memory and generates the actual signal that draws the picture on the display.

Diagram: Raster Display Architecture



Key Concepts

- **Resolution**: Total number of pixels (e.g., 1920×1080) — determines frame buffer size.
- **Bit Depth / Bits per Pixel (bpp)**: Number of bits used to store each pixel's color — e.g., 24 bpp allows 2^{24} (about 16.7 million) distinct colors.
- **Refresh Rate**: Number of times per second the video controller redraws the ENTIRE screen from the frame buffer (e.g., 60 Hz = 60 times/second) — must be fast enough to avoid visible flicker.
- **Frame Buffer Size Formula**: Total memory required = Resolution (total pixels) × Bits per Pixel.

Worked Example

For a screen of resolution 1024 × 768 with 8 bits per pixel:

Frame Buffer Size = $1024 \times 768 \times 8 \text{ bits} = 6,291,456 \text{ bits} = \mathbf{786,432 \text{ bytes} \approx 768 \text{ KB}}$

Exam Tip

Frame buffer SIZE calculation (Resolution × bpp) is a common numerical sub-part within this topic — always show the multiplication explicitly and convert the final answer to KB/MB for a complete answer.

4. Line Generating Algorithms (DDA & Bresenham's)

In Simple Words

Since a screen is made of discrete pixels, drawing a mathematically "straight" line actually means choosing WHICH pixels to light up to best approximate that line. Line generating algorithms decide exactly which pixels to plot.

DDA (Digital Differential Analyzer) Algorithm

- Calculates pixel positions using the line's slope, incrementing by 1 unit in the dominant direction (X if slope ≤ 1 , Y if slope > 1) and by the slope amount in the other direction, then ROUNDING to the nearest integer pixel.
- Simple to implement, but uses floating-point arithmetic and rounding at every step — slower and can accumulate small errors.

Bresenham's Line Algorithm

- Uses ONLY integer arithmetic (addition, subtraction, and bit-shifting) by tracking a Decision Parameter that determines whether the next pixel should move straight or diagonally.
- Much faster than DDA (no floating-point, no rounding operations) — the standard algorithm used in real graphics hardware/libraries.

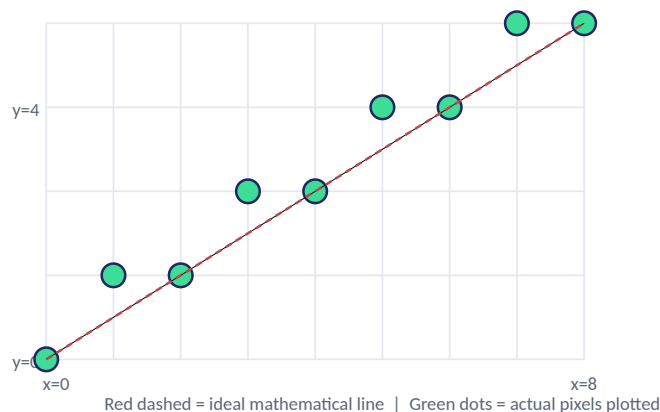
Bresenham's Decision Parameter (for slope 0 to 1)

Initial: $p_0 = 2\Delta y - \Delta x$

If $p_k < 0$: next pixel is (x_k+1, y_k) , and $p_{k+1} = p_k + 2\Delta y$

If $p_k \geq 0$: next pixel is (x_k+1, y_k+1) , and $p_{k+1} = p_k + 2\Delta y - 2\Delta x$

Diagram: Bresenham's Line from (0,0) to (8,4)



Solved Example Result

For endpoints (0,0) to (8,4): $\Delta x=8$, $\Delta y=4$, $p_0=2(4)-8=0$.

Plotted pixels in order: (0,0), (1,1), (2,1), (3,2), (4,2), (5,3), (6,3), (7,4), (8,4) — forming a clean "staircase" that best approximates the true line.

Exam Tip

Bresenham's numerical questions ALWAYS ask you to tabulate p_k , decision, and next pixel for every step — draw a table with columns (k, p_k , Decision, Next Pixel) rather than just listing final coordinates, since the step-by-step working carries most of the

marks.

5. Circle Generating Algorithm (Midpoint Circle)

In Simple Words

Just like lines, circles must also be approximated using discrete pixels. The Midpoint Circle Algorithm (Bresenham's Circle Algorithm) efficiently decides which pixels to plot using only integer arithmetic, exploiting the circle's 8-way symmetry.

Eight-Way Symmetry

A circle is symmetric across both axes AND both diagonals. So the algorithm only needs to compute pixel positions for ONE octant (45°) — the remaining 7 octants are found instantly by reflecting the coordinates (swapping and negating x, y as needed).

Midpoint Circle Decision Parameter

Initial: $p_0 = 1 - r$ (where r is the radius)

If $p_k < 0$: next point is (x_k+1, y_k) , and $p_{k+1} = p_k + 2x_{k+1} + 1$

If $p_k \geq 0$: next point is (x_k+1, y_k-1) , and $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$

The 8 Symmetric Points for Any Computed (x, y)

(x, y)	(-x, y)	(x, -y)	(-x, -y)
(y, x)	(-y, x)	(y, -x)	(-y, -x)

Solved Example — Circle of Radius 10, Starting Point

$p_0 = 1 - 10 = -9$ (negative) $\rightarrow$ next point = $(x+1, y) = (1, 10)$, and all 8 symmetric points are plotted immediately: $(1,10), (-1,10), (1,-10), (-1,-10), (10,1), (-10,1), (10,-1), (-10,-1)$.

Exam Tip

Always explicitly state and use the 8-way symmetry property — computing only 1/8th of the circle and mentioning "the remaining 7 points follow by symmetry" is what separates a complete answer from a partial one, even before showing the full numeric trace.

6. 2D Transformations

In Simple Words

Transformations let us move, resize, or rotate a shape mathematically. Each basic transformation (translation, scaling, rotation) is represented as a matrix, so multiple transformations can be COMBINED by simply multiplying their matrices together.

Basic 2D Transformations

Transformation	Effect	Formula (new coordinates)
Translation	Shifts a point by (tx, ty)	$x' = x + tx, \quad y' = y + ty$
Scaling	Resizes relative to origin by factors (sx, sy)	$x' = x \cdot sx, \quad y' = y \cdot sy$
Rotation	Rotates a point by angle θ about the origin	$x' = x \cdot \cos\theta - y \cdot \sin\theta, \quad y' = x \cdot \sin\theta + y \cdot \cos\theta$

Homogeneous Coordinates

Why We Need Them

Ordinary 2D transformations mix ADDITION (translation) with MULTIPLICATION (scaling/rotation), which cannot be combined into a single matrix multiplication. Homogeneous coordinates solve this by representing a 2D point (x,y) as a 3D vector (x,y,1) — this allows even translation to be expressed as matrix multiplication, so ALL transformations can be chained via simple matrix products.

Matrix Forms (Homogeneous)

Translation Matrix

```
| 1 0 tx |  
| 0 1 ty |  
| 0 0 1 |
```

Scaling Matrix

```
| sx 0 0 |  
| 0 sy 0 |  
| 0 0 1 |
```

Rotation Matrix (by angle θ)

```
| cos $\theta$  -sin $\theta$  0 |  
| sin $\theta$  cos $\theta$  0 |  
| 0 0 1 |
```

Composite Transformations

Multiple transformations applied in sequence (e.g., "scale, then rotate, then translate") are combined into ONE final matrix by multiplying the individual matrices together IN ORDER — the combined matrix can then be applied once to every point of the object, which is far more efficient than applying each transformation separately.

Exam Tip

Always mention WHY homogeneous coordinates are needed (to unify translation with scaling/rotation into pure matrix multiplication) — this reasoning is frequently asked as a standalone question, not just the formulas.

7. Windowing and Clipping

In Simple Words

A "Window" is the rectangular region of your scene you want to view; the "Viewport" is where that window is displayed on the actual screen. Clipping is the process of cutting away any part of the picture that falls OUTSIDE the window, so only the visible portion is drawn.

Window-to-Viewport Mapping

Formula

For a point (x_w, y_w) in the window, mapped to the viewport:

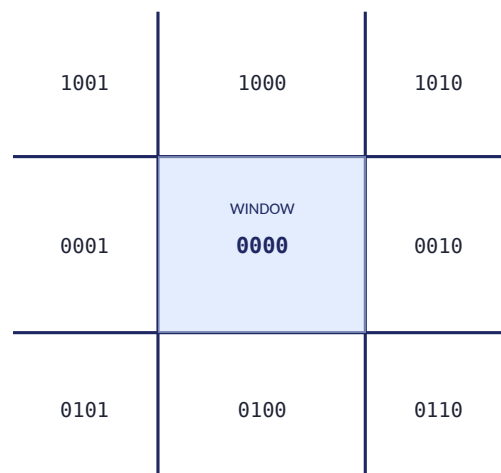
$$x_v = x_{vmin} + (x_w - x_{wmin}) \times (\text{Viewport Width} / \text{Window Width})$$

$$y_v = y_{vmin} + (y_w - y_{wmin}) \times (\text{Viewport Height} / \text{Window Height})$$

Cohen-Sutherland Line Clipping Algorithm

Assigns each endpoint a 4-bit Region Code based on its position relative to the window (Top, Bottom, Right, Left), then uses simple bitwise tests to quickly classify and clip lines.

Diagram: Cohen-Sutherland Region Codes



Reading the Codes (bit order: Top, Bottom, Right, Left)

Center region (the visible window) = 0000. A point above and to the right of the window gets code 1010 (Top=1, Right=1). If BOTH endpoints' codes AND together give non-zero, the line is completely OUTSIDE and trivially rejected. If BOTH codes are 0000, the line is completely INSIDE and trivially accepted. Otherwise, the line is clipped against the window edges one at a time.

Sutherland-Hodgman Polygon Clipping

Clips a polygon (not just a single line) against each edge of the window ONE AT A TIME — the polygon is successively clipped against the left, right, top, and bottom boundaries, with each stage potentially adding new vertices where the polygon boundary crosses the clip edge.

Exam Tip

Draw the 9-region grid with bit codes (as shown) for ANY Cohen-Sutherland question, even if not explicitly asked — it's the fastest way to demonstrate you understand the trivial accept/reject logic before doing the detailed clipping math.

8. Three-Dimensional Graphics

In Simple Words

3D Graphics adds a Z coordinate (depth) to represent real-world objects, and since screens are flat (2D), a Projection process is needed to "flatten" the 3D scene onto the 2D viewing plane.

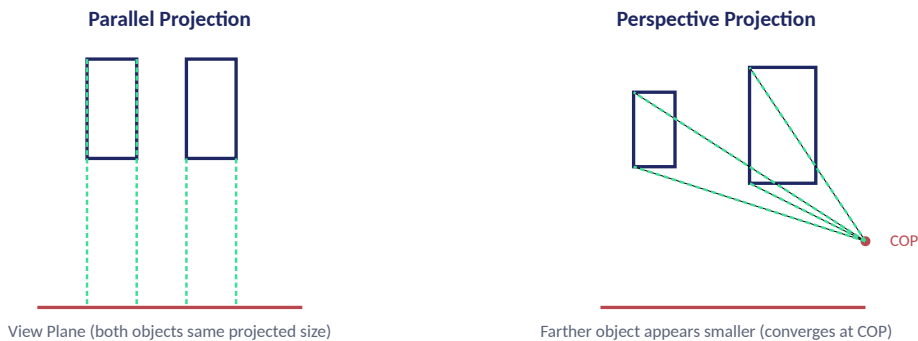
3D Transformations

Translation, Scaling, and Rotation all extend naturally to 3D using 4x4 homogeneous matrices (adding a Z row/column) — but Rotation now needs to specify WHICH axis (X, Y, or Z) the rotation happens around, since 3D rotation is axis-dependent.

Types of Projections

Projection Type	Description
Parallel Projection	Projection lines are parallel to each other; preserves relative proportions but looks less "realistic." Sub-types: Orthographic (perpendicular to viewing plane) and Oblique.
Perspective Projection	Projection lines converge at a single point (Center of Projection); objects farther away appear smaller — mimics how human eyes/cameras actually see the world, giving a realistic sense of depth.

Diagram: Parallel vs Perspective Projection



Exam Tip

Always state that Perspective Projection is more REALISTIC (matches human vision) while Parallel Projection is more useful for ENGINEERING/technical drawings (preserves true measurements/proportions) — this practical distinction is what examiners want beyond just the definitions.

9. Curves and Surfaces

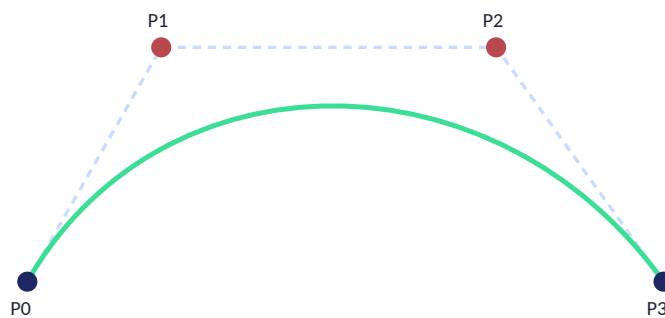
In Simple Words

Instead of building curved shapes from many tiny straight lines, Computer Graphics uses mathematical curve models — defined by a small number of "control points" — that can generate smooth curves and surfaces efficiently.

Bezier Curves

- Defined by a set of Control Points; the curve always passes through the FIRST and LAST control points, but is only "pulled toward" the intermediate ones (doesn't pass through them).
- A Cubic Bezier Curve uses exactly 4 control points (P0, P1, P2, P3).
- The shape of the "control polygon" (line connecting the control points in order) gives a good visual approximation of the curve's shape.

Diagram: Cubic Bezier Curve



Reading This Diagram

The curve (green) passes exactly through P0 and P3 (the endpoints), but only bends TOWARD P1 and P2 without touching them — this "pull without touching" behavior is the defining visual signature of a Bezier curve.

B-Spline Curves

An extension of Bezier curves that use a series of connected curve segments, each controlled by only a LOCAL subset of control points. This gives Local Control — moving ONE control point only affects the nearby part of the curve, unlike a Bezier curve where moving any control point can affect the ENTIRE curve's shape.

Exam Tip

When comparing Bezier vs B-Spline, always highlight the LOCAL vs GLOBAL control difference — this single property is the most commonly tested distinction between the two curve types.

10. Hidden Line and Surface Removal

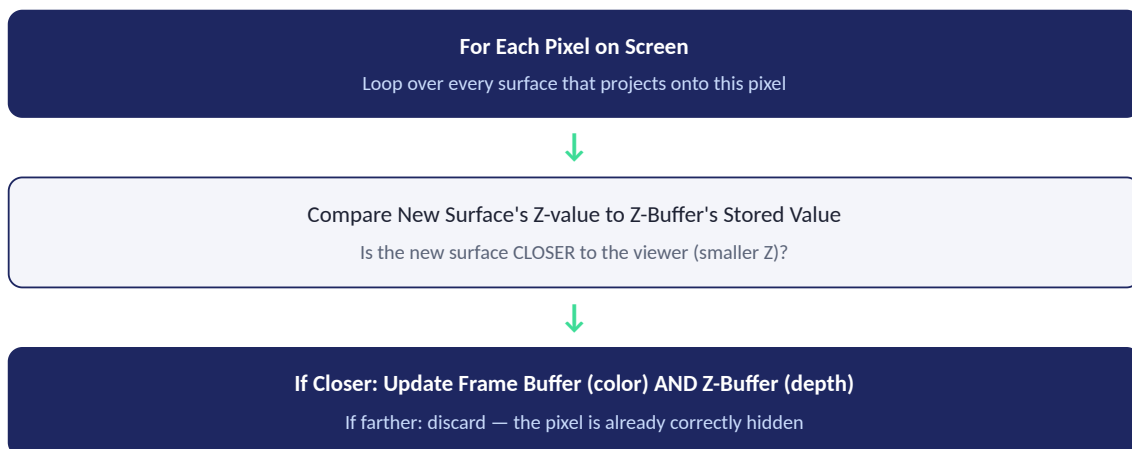
In Simple Words

When rendering a 3D scene, some lines/surfaces are blocked from view by closer objects. Hidden Line and Surface Removal algorithms figure out which parts should NOT be drawn, so the final image looks correct (like a real, solid object) rather than showing "X-ray" views through everything.

Major Techniques

- **Z-Buffer (Depth-Buffer) Algorithm:** Maintains a second buffer storing the depth (Z value) of the closest surface drawn so far, for EVERY pixel. When a new surface is rendered, its depth is compared to the stored value — only drawn if it's CLOSER to the viewer than what's already there. Simple, and works on any scene, but requires extra memory for the depth buffer.
- **Painter's Algorithm:** Sorts all surfaces by depth (farthest first) and draws them in that order, so nearer surfaces naturally overwrite/hide farther ones — like a painter painting the background first, then foreground objects on top. Fails for certain overlapping/cyclic surface arrangements.
- **Back-Face Culling:** A quick optimization — surfaces facing AWAY from the viewer (back faces of a solid, opaque object) can never be visible, so they are discarded immediately without further processing, reducing the workload for later hidden-surface steps.

Diagram: Z-Buffer Comparison Per Pixel



Exam Tip

Always name all THREE techniques (Z-Buffer, Painter's, Back-Face Culling) even if the question only asks about one — mentioning that Z-Buffer is the most commonly used in modern GPU hardware (due to its simplicity and generality) is a strong closing point for full marks.

— End of Notes —

Computer Graphics | UP LT Grade CS Mains Preparation